Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 2

Linux Navigation and File Management

Lab 2 guide: concepts first, then move and copy files with your own paths.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Move around with absolute and relative paths, and explain . and ..
2. Build a folder tree, then move and copy files without changing their contents.
3. Find the cause of a path error and prove that a copy is identical.

THIS LAB

Lab 2

First the concepts, with examples. Then the lab on the server: Core for everyone, Plus and Challenge if you finish early, and a live checkpoint.

DISCUSSION

How do you tell a friend exactly where a file is?

From the top of the disk, or from where your friend is now? Linux has a name for each: an absolute path and a relative path.

CONCEPTS FIRST

Files, folders and paths

The ideas this lab uses, each with an example.

Everything is a file

- Files store data: text, pictures, programs.
- A directory is a special file that holds other files. Windows and macOS call it a folder. The two words mean the same thing.
- Directories inside directories make a tree.
- Many Linux servers have no desktop, so you manage files from the command line.

FOR THE WHOLE COURSE

Why learn it here?

Every later lab works in folders on the server. If you can say where a file is, you can reach it from anywhere.

One tree, not one per drive

Windows: one tree for each drive

Windows

```
This PC
├── C:
│   ├── Program Files
│   ├── Users
│   └── Windows
├── D:
│   └── Data
└── E: (USB stick)
    └── Photos
```

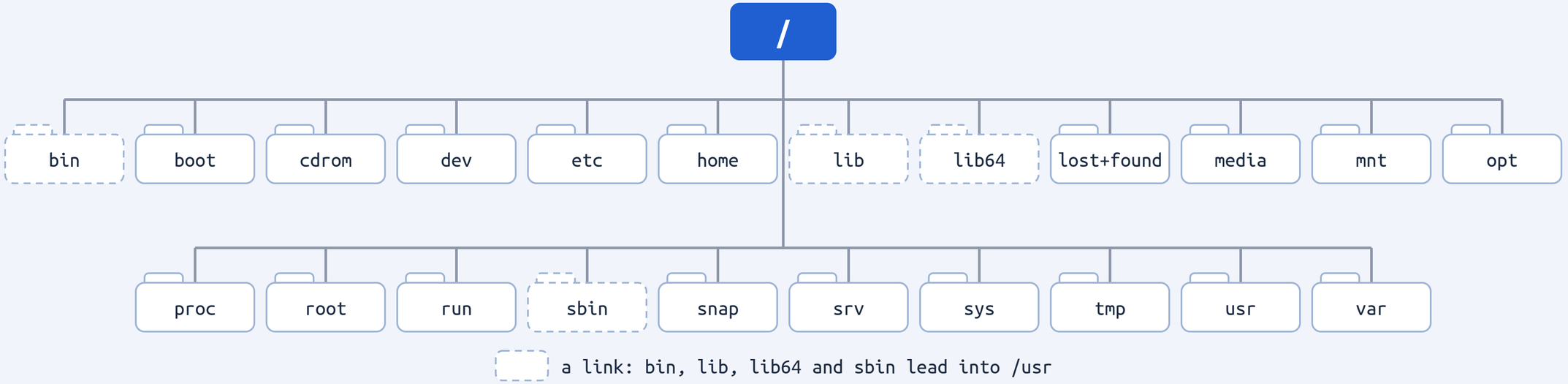
Linux: one tree that starts at /

Linux

```
/
├── bin
├── etc
├── home
│   └── student
├── media
│   └── usb
├── tmp
└── usr
```

The top of the Linux tree is the root, written /. A USB stick appears as a folder in the same tree, for example under /media.

Every folder in /, with the root at the top



This is ls / on the course server. Ubuntu made all of them when it was installed; the next three slides say what each one is for.

What each folder is for (1 of 3)

In /	What it is for
/bin	Link to /usr/bin: the programs everybody uses, such as ls, cp and mv
/boot	The kernel and the files that start the system
/cdrom	Where an installation disc appears. Left over from installing Ubuntu
/dev	Devices shown as files: disks, terminals, /dev/null
/etc	Settings of the system (configuration files), for example /etc/passwd
/home	One home folder for each user
/lib, /lib64	Links to /usr/lib and /usr/lib64: libraries that programs need
/lost+found	Pieces of files rescued by a disk check. Only root can open it

What each folder is for (2 of 3)

In /	What it is for
`/media`	USB sticks and DVDs appear here when they are plugged in
`/mnt`	A place where the administrator attaches a disk for a while
`/opt`	Optional software installed outside the package manager
`/proc`	Not on the disk: the kernel shows running processes here, for example /proc/cpuinfo
`/root`	The home of the administrator account, root. It is not the root /
`/run`	Information about the system since it started. Kept in memory
`/sbin`	Link to /usr/sbin: programs for the administrator

What each folder is for (3 of 3)

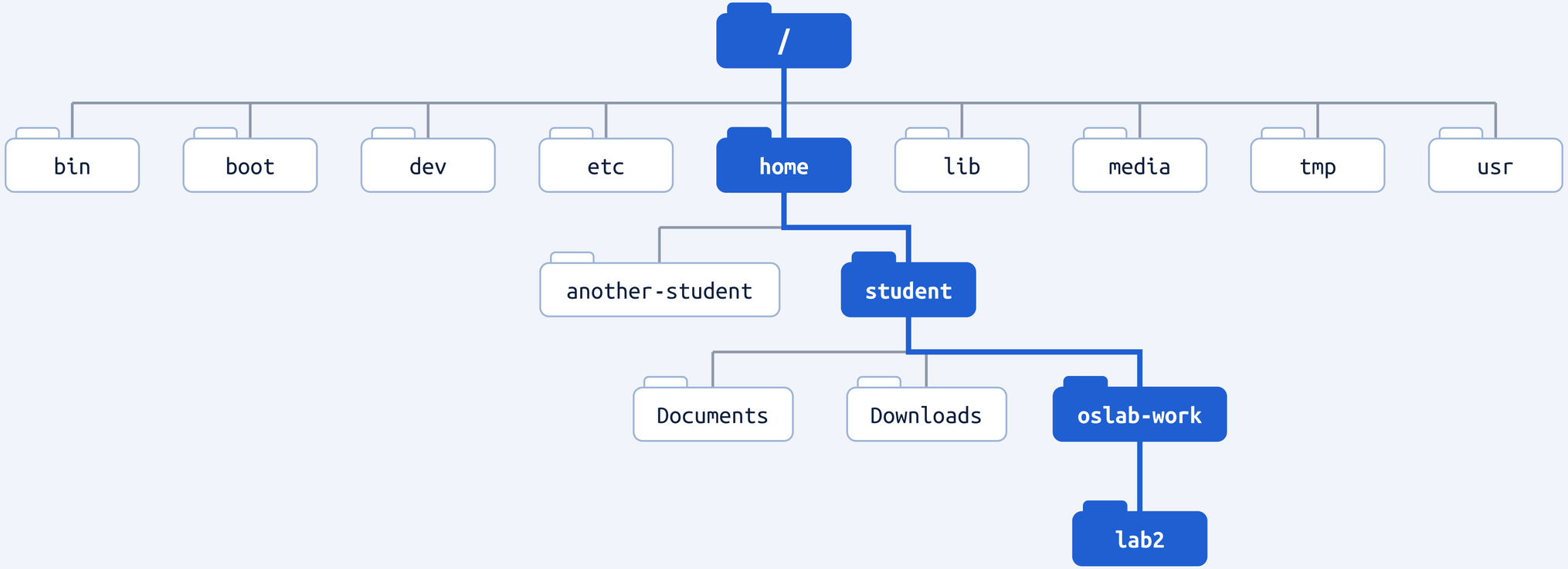
In /	What it is for
`/snap`	Software installed as snap packages (Ubuntu)
`/srv`	Data that this server offers, for example a website
`/sys`	Not on the disk: the kernel shows devices and its settings here
`/tmp`	Temporary files. Every user may write here. Emptied at restart
`/usr`	Installed software: programs in /usr/bin, libraries in /usr/lib, documents in /usr/share
`/var`	Data that changes: logs in /var/log, mail, caches
`swap.img`	A file, not a folder: disk space used as extra memory (swap)

The folders you meet in this course

Folder	What it holds
`/`	The root: everything is below it
`/bin`	Programs everybody can run: ls, cp, mv and the others
`/etc`	Settings of the system (configuration files)
`/home`	One home folder for each user
`/tmp`	Temporary files, deleted at restart
`/usr`	Installed software
`/media`	Disks and USB sticks

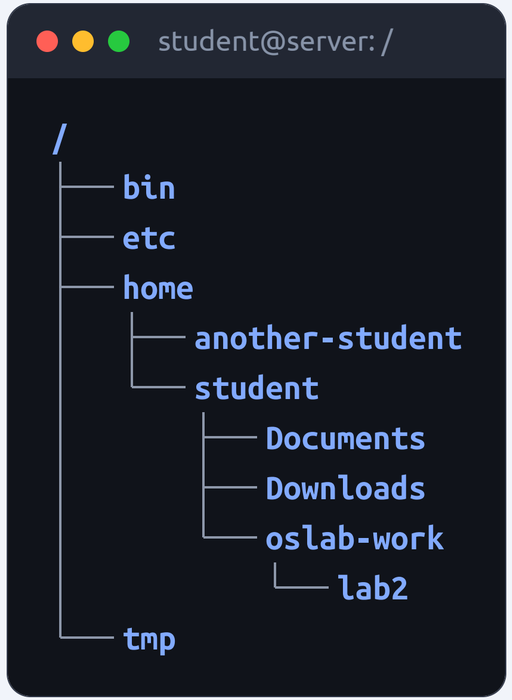
ls / shows the real list. It is longer; these are the ones you need now.

The tree, with the root at the top



Follow the blue line down from the root: it is the path /home/student/oslab-work/lab2, your lab folder.

Your home directory



Every account has its own home: /home/ followed by the account name. When you log in, you start there.

You write	It means
`/home/student`	The full name of the home of student
`~`	Your own home, whoever you are
`$HOME`	The same, as a shell variable

You may create files only in your home and in /tmp. Your lab folder is ~/oslab-work/lab2.

The prompt shows where you are

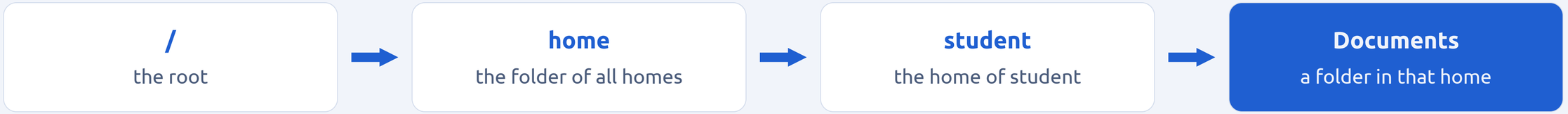


The prompt shows ~ while you are at home, and the real folder when you are somewhere else.

pwd prints the full name of the folder you are in.

cd with nothing after it takes you home from anywhere.

What is a path?



The path

/home/student/Documents: the folders you walk through, from the left.

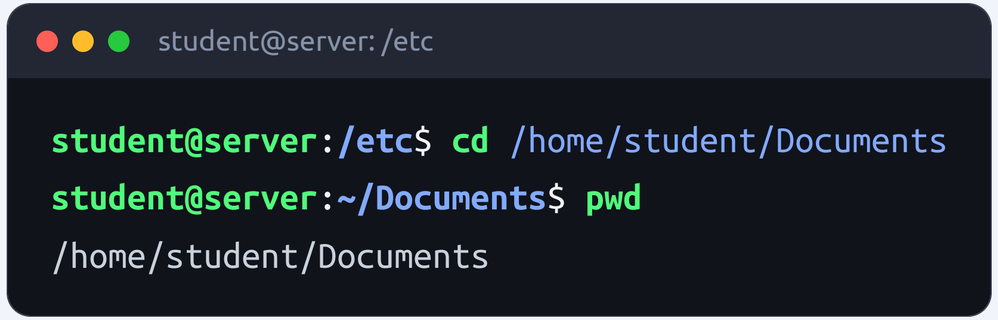
The / sign

At the start it is the root. Between two names it separates one folder from the next.

A file at the end

/home/student/Documents/cv.txt is the file cv.txt in the folder Documents.

Absolute paths start at the root



An absolute path starts with /.

It starts at the root, so it means the same place from anywhere: here the command works from /etc, and it would work the same from any other folder.

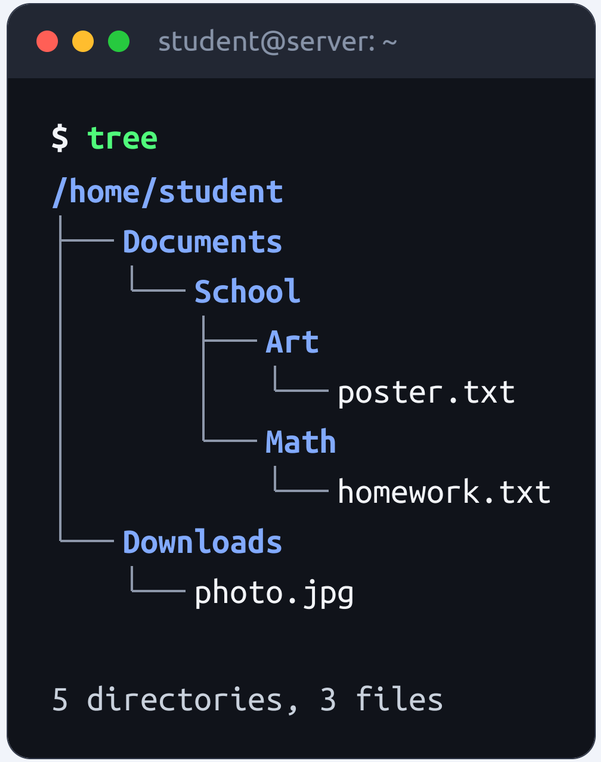
Relative paths start where you are



A relative path does not start with /.

It starts from the folder you are in now, which pwd shows. From your home, Documents is there. From /etc, it is not.

An example tree



The next slides use this tree in the home of student.

It is not the lab's tree. In the lab you use the same ideas on your own folders.

The same files, from different folders

You are in (pwd)	You want	Absolute path	Relative path
/home/student	poster.txt	/home/student/Documents/School/Art/poster.txt	Documents/School/Art/poster.txt
/home/student/Documents/School	poster.txt	/home/student/Documents/School/Art/poster.txt	Art/poster.txt
/home/student/Documents/School/Math	poster.txt	/home/student/Documents/School/Art/poster.txt	../Art/poster.txt
/home/student/Documents/School/Art	photo.jpg	/home/student/Downloads/photo.jpg	../../../Downloads/photo.jpg

The absolute path is the same in every row. The relative path changes, because it starts from where you are.

Two dots: one level up

student@server: ~

```
/home/student
├── Documents
│   └── School
│       ├── Art
│       │   └── poster.txt
│       └── Math
│           └── homework.txt
└── Downloads
    └── photo.jpg
```

you start in Art

```
student@server:~/Documents/School/Art$ cd ..
student@server:~/Documents/School$ pwd
/home/student/Documents/School
student@server:~/Documents/School$ cd ../../Downloads
student@server:~/Downloads$ pwd
/home/student/Downloads
```

Each .. goes up one folder from where you are now. From School, ../.. is /home/student.

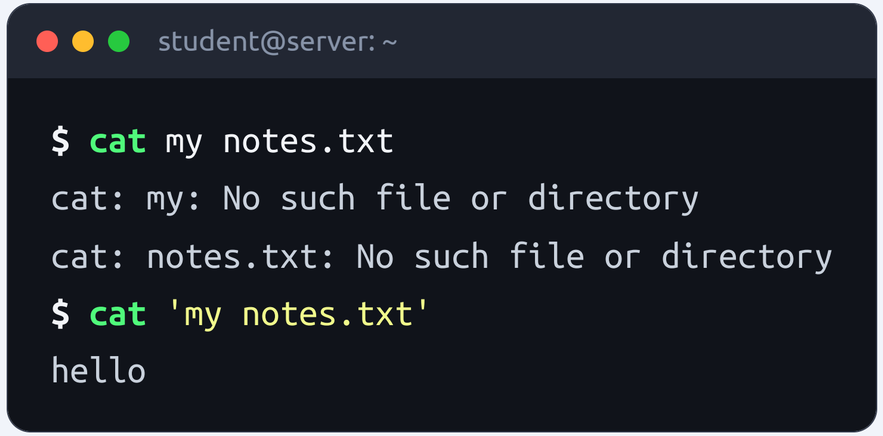
.. does not mean your home or the lab folder. It always counts from the folder you are in.

Four special names

Name	Means	Example
`.`	The folder you are in now	`cp /tmp/a.txt .` copies a.txt here
`..`	The folder one level above	`cd ..` goes up one level
`../..`	Two levels above	`cd ../..` goes up two levels
`~`	Your home	`cd ~/oslab-work`

ls -a shows . and .. as the first two lines of every folder.

Names with spaces



The shell splits a command into words at every space. Without quotes, cat receives two names.

With quotes, the name is one word again.

Double quotes still replace a variable: "$file1" is your file name. Single quotes keep the text exactly as typed.

In this lab you also see -- before file names, as in cp -- "$file1" reports/. It means: no more options after this point.

File and folder commands

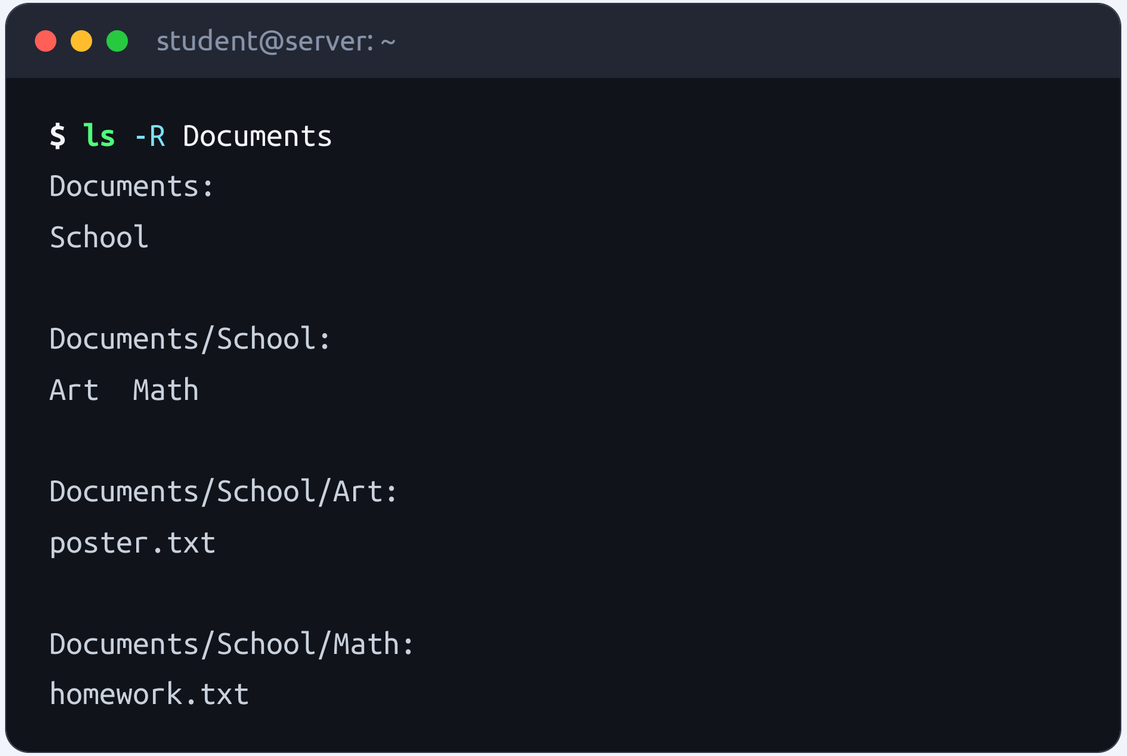
Command	Meaning
`pwd`	Prints the folder you are in
`ls`	Lists a folder
`cd`	Changes folder
`mkdir -p`	Makes a chain of folders
`touch`	Makes an empty file
`cat`	Shows what is in a file
`find -type f`	Lists the files below a folder

Command	Meaning
`cp`	Copies a file. The original stays
`cp -r`	Copies a folder with all inside
`mv`	Moves or renames a file
`rm`	Deletes a file. No recycle bin
`rmdir`	Deletes an empty folder
`cmp`	Compares two files byte by byte
`nano`	A text editor in the terminal

When the destination of cp or mv is a folder, the file keeps its name inside it. When it is a new name, the file gets that name.

Listing options

You type	It shows
`ls -l`	Long list: permissions, size, date
`ls -a`	Also hidden names, which start with .
`ls -R`	Every folder below, too (recursive)
`ls -lt`	Newest first
`ls -lS`	Largest first
`ls -lh`	Sizes like 4.0K
`ls -ltr`	-r reverses the order: oldest first



Try them together

student@server: ~

```
student@server:~$ mkdir -p demo/School/Art demo/School/Math
student@server:~$ cd demo/School
student@server:~/demo/School$ echo 'draft' > Art/poster.txt
student@server:~/demo/School$ cp Art/poster.txt Math/
student@server:~/demo/School$ mv Math/poster.txt Math/copy.txt
student@server:~/demo/School$ cmp Art/poster.txt Math/copy.txt
student@server:~/demo/School$ cd ../..
```

the result

```
$ tree demo/School
School
├── Art
│   └── poster.txt
└── Math
    └── copy.txt

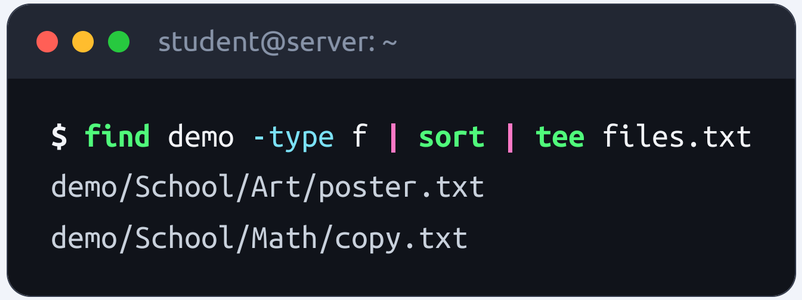
2 directories, 2 files
```

cmp printed nothing, so the copy is identical.

Delete the practice folder with rm -r demo. rm -r deletes a folder and everything in it, with no undo: run pwd first.

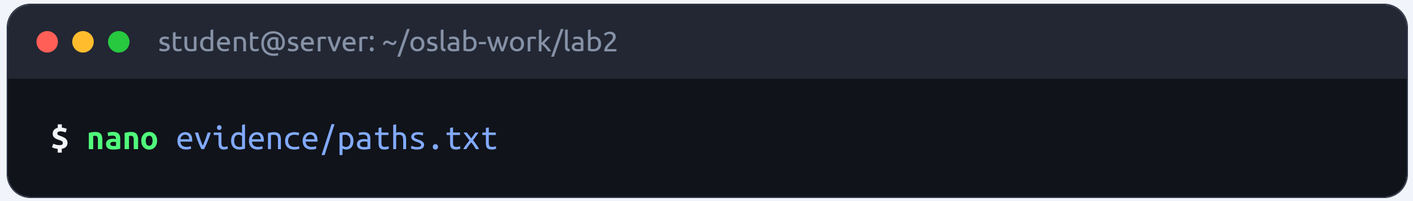
Sending output somewhere else

You type	What happens
`ls > list.txt`	Saves the output in the file, replacing what was there (Lab 1)
`ls >> list.txt`	Adds the output to the end of the file (Lab 1)
`ls | sort`	The pipe passes the output to the next command; sort puts the lines in order
`ls | tee list.txt`	tee shows the output on the screen and also saves it in the file



Read it from left to right: find lists the files, sort puts them in order, tee shows the list and saves it. You use exactly this in Core 3.

Writing a file with nano



nano is a simple text editor in the terminal. You use it in Core 2 to keep the commands that worked.

The keys are listed at the bottom of the nano screen. The sign ^ means Ctrl.

Press	To
Type or paste	Write your text
`Ctrl+O`, then `Enter`	Save the file
`Ctrl+X`	Leave nano

LAB 2

Navigation and file management

Two reports arrived in TechCorp's incoming folder. Put them in the right places, unchanged.

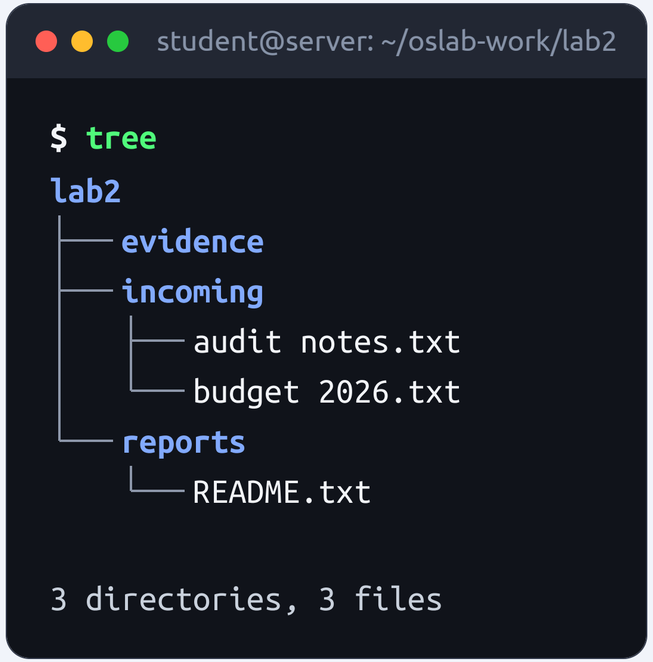
The scenario

SCENARIO

Your job at TechCorp

Put the two reports in the right places in the company's folders, keep their contents unchanged, and be able to reach them from any department.

Your two file names are only yours. This guide shows example names.



Inside the 120 minutes

Minutes	What happens
0 to 5	Setup
5 to 20	Concepts, then guided: where am I?
20 to 30	Prediction, then class discussion
30 to 40	Core 1: build the company folders
40 to 65	Core 2: move and copy with your own paths

Minutes	What happens
65 to 75	Core 3: prove it
75 to 90	Plus and Challenge, or finish Core
90 to 105	Live checkpoint
105 to 115	Debrief
115 to 120	Submit

Six helper commands, and why you use them

Command	What it does	Why
`oslab values lab2`	Shows your file names and folders	Your work is different from your neighbour's
`oslab predict lab2`	Saves your prediction once	Guessing first makes you think. The first answer is kept
`oslab hint lab2 1`	A hint, levels 1 to 3	You get unstuck without waiting, at no cost
`oslab check lab2`	Looks at your folders, shows milestones	You see at once what is still missing
`oslab checkpoint lab2`	A short test near the end	It shows what you can do alone
`oslab debrief lab2`	After the lab: the right answers to your prediction	It shows what you corrected, and it is marked

How you work

Core, Plus, Challenge

Core is for everyone. Plus and Challenge are for students who finish early. Full marks need only Core and the checkpoint.

AI tools

Allowed in Core, Plus and Challenge, so test what they tell you. Not allowed in the prediction and the checkpoint.

Core 2

Nobody gives you the commands there. You write every path yourself. Hints cost no points.

A wrong prediction costs nothing. It is marked for being made and corrected, not for being right.

Setup: start the lab

student@server: ~

```
$ export OSLAB_WORKSPACE="${OSLAB_WORKSPACE:-$HOME/oslab-work}"
$ oslab start lab2
$ cd "$OSLAB_WORKSPACE/lab2"
$ mkdir -p evidence
$ oslab values lab2
```

- The first line sets the variable OSLAB_WORKSPACE to ~/oslab-work, unless it is already set. export lets oslab see it. Type it as it is.
- oslab start lab2 makes the lab folder with your two reports.
- cd takes you into it.
- oslab values lab2 prints your own names. The next slide shows what to do with them.

Your values, kept in variables

1. The command prints your values

example: your values will be different

```
$ oslab values lab2
Your personal values for lab2 (your-account):
  file1    = budget 2026.txt
  file2    = audit notes.txt
  owner    = Finance
  reader   = HR
  third    = Support
```

2. Type them into shell variables

type your own values

```
$ file1='budget 2026.txt'
$ file2='audit notes.txt'
$ owner=Finance
$ reader=HR
$ third=Support
```

Write name=value with no spaces around =. Keep the quotes: your file names have a space. In a new terminal, set them again.

Variables and placeholders

The instruction says	With the example values, you type	Or, with the variable
`ls -l "incoming/$file1"`	`ls -l 'incoming/budget 2026.txt'`	`ls -l "incoming/$file1"`
`TechCorp/<owner>/`	`TechCorp/Finance/`	`"TechCorp/$owner/"`
`TechCorp/<reader>/`	`TechCorp/HR/`	`"TechCorp/$reader/"`

"$file1"

The shell replaces it with your first file name, and the double quotes keep it one word.

<owner>

A placeholder: never type the angle brackets. Type your own value, or the variable in double quotes.

GUIDED

Where am I?

Type along with the instructor.

Steps 1 and 2: look, then practise

student@server: ~/oslab-work/lab2

```
$ pwd
$ ls
$ ls -la

$ mkdir -p practice/notes
$ echo 'orientation' > 'practice/welcome note.txt'
$ cd practice
$ pwd
$ cp -- 'welcome note.txt' notes/
$ cat 'notes/welcome note.txt'
$ cd ..
$ pwd
```

pwd prints an absolute path: it starts with /.

In ls -la, the first two lines are . and .., this folder and the one above.

echo with > writes the text into a new file. -- after cp means: no more options. cd .. goes to the parent.

Steps 3 and 4: two paths, and the quotes

3. One file, two paths

student@server: ~/oslab-work/lab2

```
$ cat 'practice/welcome note.txt'
$ cat "$OSLAB_WORKSPACE/lab2/practice/welcome note.txt"
```

Which of the two stops working if you cd somewhere else?

4. Why the quotes matter

student@server: ~/oslab-work/lab2

```
$ ls -l "incoming/$file1"
$ ls -l incoming/$file1
```

The second command fails. Read its error: which names did ls receive?

Without the double quotes, the shell splits the name at its space, so ls looks for two files that do not exist.

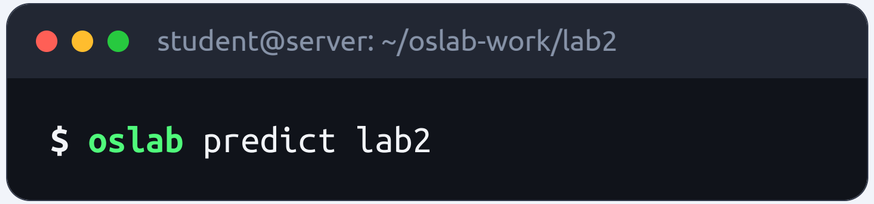
PREDICT

Guess first, then try

Five questions, before you move any report.

Five questions, before the experiment

1. You are in incoming/. Type the relative path of the reports directory.
2. You change to the lab folder. Does that path still point to reports?
3. After mv 'incoming/<file1>' reports/, how many files with that name are in the lab folder tree?
4. Then after cp 'reports/<file1>' TechCorp/<owner>/, how many are there?
5. One sentence: why did you answer like this?



PREDICTION

The rules

Alone, and without AI. The first answer is the one that counts. A wrong guess costs nothing.

The class then looks at the spread of answers together, without names.

CORE 1

Build the company folders

Ten minutes. Then you read the two reports.

Make the departments, read the reports

student@server: ~/oslab-work/lab2

```
$ cd "$OSLAB_WORKSPACE/lab2"
$ mkdir -p "TechCorp/$owner/archive" "TechCorp/$reader" "TechCorp/$third"
$ find TechCorp -type d
$ cat "incoming/$file1" "incoming/$file2"
```

with the example values

```
$ tree TechCorp
TechCorp
├── Finance
│   └── archive
├── HR
└── Support

4 directories, 0 files
```

find -type d lists only folders. cat with two names shows one file, then the other: write down what they contain.

CORE 2

Move and copy with your own paths

Nobody gives you the commands here. You write every path yourself.

Six steps

1. Go into incoming/. Move your first file to reports/ with a relative destination.
2. Go back to the lab folder. Move your second file to reports/ with absolute paths for both source and destination.
3. Copy both reports from reports/ into TechCorp/<owner>/. The originals stay in reports/.
4. Copy your first report into TechCorp/<owner>/archive/ as first-original.txt.
5. Go into TechCorp/<reader>/. Read the first report in the owner department: once relative, once absolute.
6. Your decision: for step 3, relative or absolute? Write which, and when the other would be better.

HABIT

Before every mv or cp

Run pwd. Then draw where you are and where the file must go.

RECORD

evidence/paths.txt

Write every command that worked into it, with nano evidence/paths.txt from the lab folder.

Absolute paths here start with "$OSLAB_WORKSPACE/lab2/...". Stuck for five minutes? oslab hint lab2 1, then 2, then 3.

Where the files must end up

student@server: ~/oslab-work/lab2

```
$ tree
lab2
├── evidence
│   └── paths.txt
├── incoming
├── reports
│   ├── audit notes.txt
│   ├── budget 2026.txt
│   └── README.txt
└── TechCorp
    ├── Finance
    │   ├── archive
    │   │   └── first-original.txt
    │   ├── audit notes.txt
    │   └── budget 2026.txt
    ├── HR
    └── Support
```

After Core 2 your folder has this shape, with your own names.

incoming/ is empty: mv took the files out of it. Your practice/ folder from the guided part is there too.

Each report is now in reports/ and in TechCorp/<owner>/: cp left the original in place.

CORE 3

Prove it

A name in ls does not prove that the contents are the same.

Step 1: prove that the copies are identical

student@server: ~/oslab-work/lab2

```
$ cd "$OSLAB_WORKSPACE/lab2"
$ cmp -- "reports/$file1" "TechCorp/$owner/$file1"
$ cmp -- "reports/$file2" "TechCorp/$owner/$file2"
```

only if a copy is wrong

```
$ cmp -- a.txt b.txt
a.txt b.txt differ: byte 1, line 1
```

You should see nothing

cmp prints nothing when two files are identical. Silence means success.

If cmp says differ

That copy is not the same. Copy it again from reports/ (Core 2, step 3) and compare again.

A name in ls does not prove the contents are the same. cmp compares the bytes.

Step 2: make one path error on purpose

example: your names will be different

```
student@server:~/oslab-work/lab2$ cd "TechCorp/$reader"
student@server:~/oslab-work/lab2/TechCorp/HR$ pwd
/home/student/oslab-work/lab2/TechCorp/HR
student@server:~/oslab-work/lab2/TechCorp/HR$ cat "reports/$file1"
cat: 'reports/budget 2026.txt': No such file or directory
```

You use the path that works from the lab folder, but you are in your reader department. The file exists, yet cat cannot find it.

Then write it down

from your reader department

```
$ nano ../../evidence/paths.txt
```

Find the path that does work from here, and test it with cat. Then add two lines at the end:

evidence/paths.txt

```
Error: cat: 'reports/budget 2026.txt': No such
       file or directory
Fix: the relative path that worked
```

After Fix: write only the path, without cat. oslab check tests both lines.

Step 3: save the list, then check

student@server: ~/oslab-work/lab2

```
$ cd "$OSLAB_WORKSPACE/lab2"
$ find incoming reports TechCorp -type f | sort | tee evidence/tree.txt
$ oslab check lab2
```

find lists the files, sort orders them, tee shows the list and saves it in tree.txt. incoming lists nothing: mv emptied it.

what oslab check lab2 should show after the list

```
PASS: both files were moved into reports/ with their contents
PASS: TechCorp/Finance/ holds identical copies
PASS: archive/first-original.txt is a copy of your first file
PASS: evidence/paths.txt records a relative and an absolute path
PASS: evidence/paths.txt has your Error: line and a Fix: path that works from your reader department
milestones: 5/5
```

A TRY line names what is still missing: fix that part and check again. Core is complete at 5/5.

If you finish early



evidence/plus.txt

Plus: find the lost file

Make the mess folders. From mess/c, without cd, move lost file.txt here as found.txt in one command. From mess/a/b, copy it to your third department with a relative path. How many .. did you need?



evidence/challenge.txt

Challenge: one command with *

* stands for any characters, so *.txt is every name ending in .txt. Copy them all into your third department with one command. Why does it fail with quotes around *.txt? Does it copy reports/.hidden.txt?

Neither is needed for full marks. They are noted in your feedback.

LIVE CHECKPOINT

Show what you can do alone

Fifteen minutes near the end of the lab. Worth 3 of the 10 points.

Why is there a checkpoint?

It shows what you can do alone

For most of the lab you may use hints and AI tools. The checkpoint is the one moment without them. It tells you, and the instructor, which paths you can really write.

Your folders are new

The starting folder and the target are made for you when the instructor opens it. An answer copied from a neighbour does not fit.

The instructor sees who needs help

Your answers arrive at once. The instructor can explain a common mistake before the next lab builds on paths.

It is fair

Everyone gets the same kind of task. You can take it even if Core is not finished.

What happens, step by step

1. The instructor opens it
Close your AI tools

→

2. oslab checkpoint lab2
Answer four questions

→

3. Read the file for real
From the new folder, save the output

→

4. oslab check lab2
A sixth milestone must pass

Before it opens

The command only says that the checkpoint is not open yet. Wait for the instructor.

While you work

Use your terminal and what you learned in this lab. No AI tools, and no help from a neighbour.

When you finish

oslab check lab2 tells you whether checkpoint.txt is right. You can fix it and check again.

What you see when you run it

example: your folders will be different

```
$ oslab checkpoint lab2
Checkpoint: your own work. Close AI tools; do not ask a neighbour.
Your current directory is `TechCorp/HR/drafts` (inside your lab2 folder; create it if it does not exist).
You must read the file `TechCorp/Finance/archive/first-original.txt` without changing directory.
Type a relative path from that directory to the file. Do not type quotes.
> your answer
How many `..` does the shortest such path need?
> your answer
From the lab folder you type  cat reports/audit notes.txt  with no quotes. How many arguments does `cat` receive?
> your answer
Which folder is `..` from that directory? Type its path from the lab folder, for example reports.
Type . if it is the lab folder itself.
> your answer
```

Type each answer after the > and press Enter. Your answers are saved once: the first answer counts.

Then read the file for real

the end of the checkpoint

```
Now do it for real from that directory and save the output:
  cat 'YOUR RELATIVE PATH' | tee "$OSLAB_WORKSPACE/lab2/evidence/checkpoint.txt"
Then run: oslab check lab2
```

You already know how

Make the starting folder with mkdir -p if it does not exist, go into it with cd, and use your own relative path.

tee shows the content and saves it in checkpoint.txt.

How the checkpoint is marked

Part	Points	Who marks it, and how
The path, the number of .., the number of arguments	1	The computer. All right is 1 point. At least half right is half a point.
Which folder .. is from your start folder	1	The computer. Each .. goes up one folder from the current one.
evidence/checkpoint.txt holds the content of the target file	1	The computer. oslab check lab2 compares it.

Can I answer again?

No. Your answers are saved once. You can still fix checkpoint.txt and run oslab check lab2 again.

Any path that works

Any relative path that reaches the target is accepted, not only the shortest one.

DEBRIEF AND SUBMIT

Look back, then hand in

Ten minutes of debrief, five to submit.

Debrief: your prediction again

student@server: ~/oslab-work/lab2

```
$ cat "$OSLAB_WORKSPACE/.records/lab2-predict.json"
$ oslab debrief lab2
```

Your prediction is in the hidden folder .records, as JSON: a text format of names and values.

oslab debrief lab2 asks each prediction question again. Type the right answer: the same one where you were right, the correct one where you were wrong. It is marked, and you may run it again.

Then the class discusses: why can a correct file name still give No such file or directory?

Submit

student@server: ~/oslab-work/lab2

```
$ SUBMISSION_REPO="$HOME/os-gic-YOUR_ID/os-lab-YOUR_ID"
$ mkdir -p "$SUBMISSION_REPO/lab2/evidence"
$ cp -- evidence/paths.txt evidence/tree.txt "$SUBMISSION_REPO/lab2/evidence/"
$ cp -- evidence/checkpoint.txt "$SUBMISSION_REPO/lab2/evidence/"
```

Copy plus.txt or challenge.txt the same way if you did them. Fill in the report template as lab2/README.md, then commit and push. The website checks these exact names.

your repository

```
$ tree
lab2
├── evidence
│   ├── checkpoint.txt
│   ├── paths.txt
│   └── tree.txt
└── README.md

1 directory, 4 files
```

Commit and push

student@server: ~

```
$ cd ~/os-gic-YOUR_ID
$ git pull
$ git add os-lab-YOUR_ID/lab2
$ git commit -m "Lab 2"
$ git push
```

As in Lab 1. git pull comes first, in case you pushed from your own computer since.

git add takes your lab2 folder, git commit records it with a message, git push sends it to GitHub.

How Lab 2 is graded

Evidence	Points
Correct folders and files: oslab check lab2 passes Core milestones 1 to 4	2
Tests: your path error and its working fix in paths.txt (milestone 5)	2
Prediction saved in time, and corrected with oslab debrief lab2	2
Live checkpoint: the three answers, the .. question and the saved output	3
Complete submission: the required files in your repository, on time	1

Every row is marked from your oslab records and your repository files, so run oslab check lab2 until it shows what you expect. Plus and Challenge are not needed for full marks.

Common mistakes

Mistake	What happens	Fix
Losing the quotes in file1='...'	The variable holds only the first word	Type it again with the quotes
$file1 without double quotes	The command receives two names	Write "$file1"
A relative path from the wrong folder	No such file or directory	Run pwd, then draw the path
Thinking .. means the lab folder	The path goes to the wrong place	Each .. is one level up from here
Proving a copy with ls	Same name, unknown content	Use cmp
Typing <owner> literally	A folder with brackets, or an error	Use your own value or "$owner"
A Fix path written from the lab folder	Milestone 5 says TRY	Write it from your reader department

Key takeaways

01

One tree from /

Every file is somewhere below the root. Your home is /home/your-account, or ~.

02

Absolute or relative

Absolute starts at /. Relative starts where you are, so check with pwd.

03

Dots

. is here. Each .. is one level up from here.

04

Copy, move, prove

cp keeps the original, mv does not. cmp proves two files are identical.

Next lab

COMING UP

Lab 3: wildcards and links

Select files with a wildcard, then create hard and symbolic links and see what happens when their target changes.

SOURCE

Reference

The Lab 2 instruction and its Concepts First section, the report template in the course repository, and man cp, man mv and man find.